DevOps Assignment 8 - Packer & Terraform

1. Installed Packer and Terraform on Mac using Homebrew, and configured AWS credentials from the AWS Academy lab session.

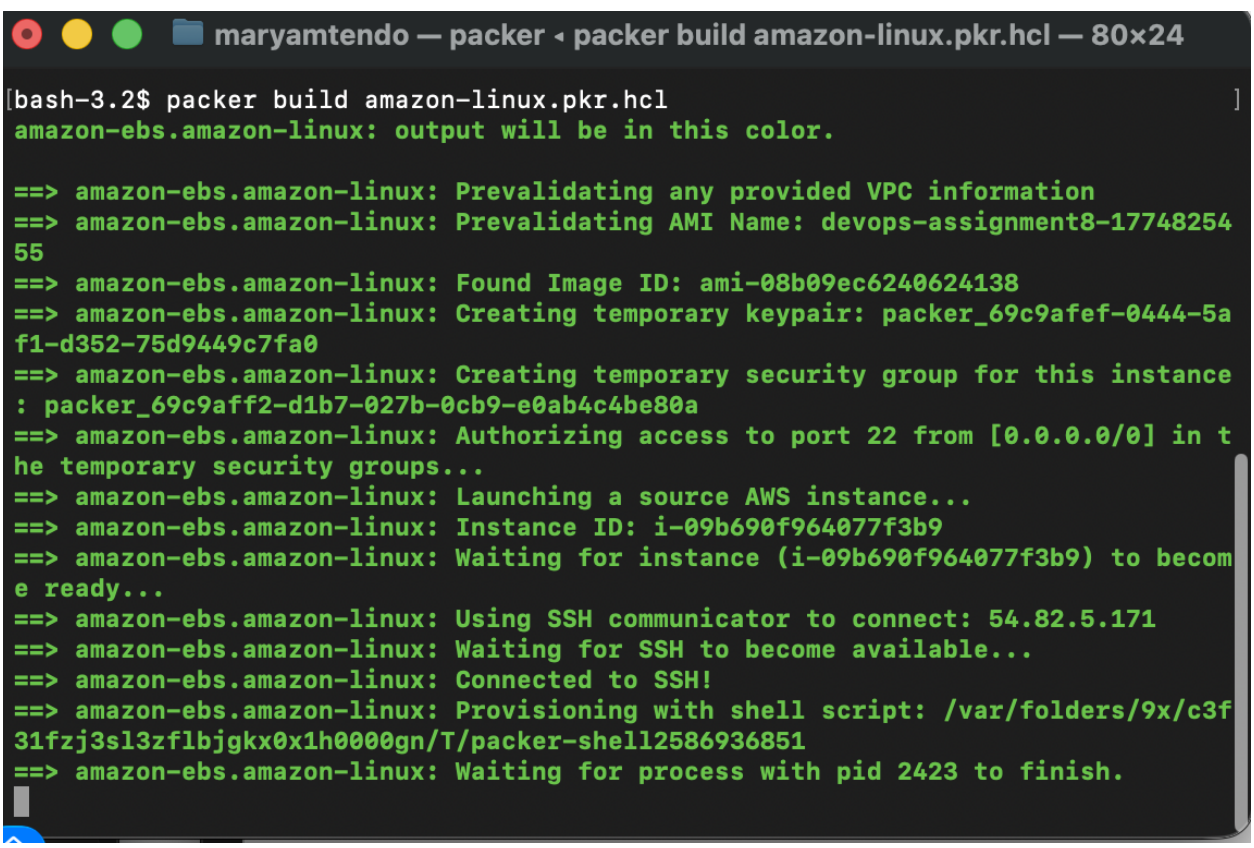
I completed the assignment is completed in two phases so that I had the AMI ready as terraform requires it to be able to launch instances.

- **Phase 1 - Packer:** I built my custom AMI (Amazon Linux ,docker my SSH public key) and was able to successfully get the AMI id.
- **Phase 2 - Terraform:** I then went ahead and used that AMI id I got in my terraform scripts to provision the VPC, subnets, bastion host, and 6 private EC2 instances.

STEPS.

I configured my AWS credentials from the lab using the AWS CLI. Then I retrieved my SSH public key by running ~/.ssh/id_rsa.pub and created a packer template file (amazon-linux.pkr.hcl). This template basically tells packer to launch a temporary Amazon Linux instance, install Docker, bake in the SSH public key, and save the result as a custom AMI.

- **Building the Custom AMI with Packer**



```
maryamtendo — bash — 80×24
==> amazon-ebs.amazon-linux: Stopping the source instance...
==> amazon-ebs.amazon-linux: Stopping instance
==> amazon-ebs.amazon-linux: Waiting for the instance to stop...
==> amazon-ebs.amazon-linux: Creating AMI devops-assignment8-1774825455 from ins
tance i-09b690f964077f3b9
==> amazon-ebs.amazon-linux: Attaching run tags to AMI...
==> amazon-ebs.amazon-linux: AMI: ami-0971f725d799840c2
==> amazon-ebs.amazon-linux: Waiting for AMI to become ready...
==> amazon-ebs.amazon-linux: Skipping Enable AMI deprecation...
==> amazon-ebs.amazon-linux: Skipping Enable AMI deregistration protection...
==> amazon-ebs.amazon-linux: Terminating the source AWS instance...
==> amazon-ebs.amazon-linux: Cleaning up any extra volumes...
==> amazon-ebs.amazon-linux: No volumes to clean up, skipping
==> amazon-ebs.amazon-linux: Deleting temporary security group...
==> amazon-ebs.amazon-linux: Deleting temporary keypair...
Build 'amazon-ebs.amazon-linux' finished after 5 minutes 3 seconds.

==> Wait completed after 5 minutes 3 seconds

==> Builds finished. The artifacts of successful builds are:
--> amazon-ebs.amazon-linux: AMIs were created:
us-east-1: ami-0971f725d799840c2

bash-3.2$
```

- Then I created variables.tf file to define all configurable values such as the AWS region, AMI ID, IP address, and instance type since I am using lab credentials that change through different sessions. So instead of hardcoding values throughout the configuration, this allows them to be defined once and referenced everywhere and easy to change too.

```
maryamtendo — nano variables.tf — 80×24
  UW PICO 5.09                          File: variables.tf

variable "region" {
  description = "AWS region"
  default     = "us-east-1"
}

variable "ami_id" {
  description = "Custom AMI ID from Packer"
}

variable "my_ip" {
  description = "Your IP address for SSH access to bastion"
}

variable "instance_type" {
  description = "EC2 instance type"
  default     = "t2.micro"
}


^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Pg   ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is  ^V Next Pg   ^U UnCut Text^T To Spell
```

- Then I created terraform.tfvars and this is where I put the actual values or credentials. This file fills in the variables I just defined in variables.tf.

```
maryamtendo — nano terraform.tfvars — 80×24
  UW PICO 5.09                    File: terraform.tfvars

region        = "us-east-1"
ami_id        = "ami-0971f725d799840c2"
my_ip         = "66.234.197.224/32"
instance_type = "t2.micro"

^G Get Help   ^O WriteOut   ^R Read File ^Y Prev Pg   ^K Cut Text   ^C Cur Pos
^X Exit       ^J Justify    ^W Where is  ^V Next Pg   ^U UnCut Text^T To Spell
```

- Then after that I created outputs.tf to define what terraform prints after provisioning is complete. This includes the bastion host public IP, the private IPs of all 6 EC2 instances, and the VPC id.

```
maryamtendo — nano outputs.tf — 80×24
  UW PICO 5.09                          File: outputs.tf

output "bastion_public_ip" {
  description = "Public IP of the bastion host"
  value       = aws_instance.bastion.public_ip
}

output "private_instance_ips" {
  description = "Private IPs of the 6 EC2 instances"
  value       = aws_instance.private[*].private_ip
}

output "vpc_id" {
  description = "VPC ID"
  value       = module.vpc.vpc_id
}

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Pg   ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is  ^V Next Pg   ^U UnCut Text^T To Spell
```

- I created a main.tf file as the core infrastructure file. This file uses the terraform-aws-modules/vpc module to provision the VPC, public subnet, private subnet, internet gateway, and NAT gateway automatically. It also defines two security groups: one allowing SSH only from the local machine's IP to the bastion host, and one allowing SSH only from the bastion to the private instances. Finally, it provisions the bastion host in the public subnet and 6 EC2 instances in the private subnet using the custom AMI built by packer.

```
maryamtendo — nano main.tf — 80×24
  UW PICO 5.09                    File: main.tf                    Modified

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.region
}

# VPC Module - creates VPC, public and private subnets, and all routing
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"

  name = "devops-assignment8-vpc"

^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Pg   ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where is  ^V Next Pg   ^U UnCut Text^T To Spell
```

- I run terraform init was run to download the AWS provider and the VPC module declared in the main.tf file, terraform apply was then run to provision all 24 AWS resources. The process took approximately 8 minutes to complete.

```
maryamtendo — terraform-provider-aws_v5.100.0_x5 ◂ terraform apply...

    }

Plan: 24 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + bastion_public_ip    = (known after apply)
  + private_instance_ips = [
      + (known after apply),
      + (known after apply),
      + (known after apply),
      + (known after apply),
      + (known after apply),
      + (known after apply),
    ]
  + vpc_id               = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

module.vpc.aws_vpc.this[0]: Creating...
```

```
maryamtendo — bash — 80×24

module.vpc.aws_nat_gateway.this[0]: Still creating... [01m40s elapsed]
module.vpc.aws_nat_gateway.this[0]: Still creating... [01m50s elapsed]
module.vpc.aws_nat_gateway.this[0]: Still creating... [02m00s elapsed]
module.vpc.aws_nat_gateway.this[0]: Creation complete after 2m6s [id=nat-0892f40
543c5f2537]
module.vpc.aws_route.private_nat_gateway[0]: Creating...
module.vpc.aws_route.private_nat_gateway[0]: Creation complete after 1s [id=r-rt
b-0a679c7fb9ae9ebdc1080289494]

Apply complete! Resources: 24 added, 0 changed, 0 destroyed.

Outputs:

bastion_public_ip = "3.91.215.61"
private_instance_ips = [
  "10.0.2.224",
  "10.0.2.113",
  "10.0.2.131",
  "10.0.2.237",
  "10.0.2.213",
  "10.0.2.42",
]
vpc_id = "vpc-0b2affb9999a6b0c4"
bash-3.2$
```

- After terraform apply completed, I used the bastion host public IP to SSH into the bastion using the private key. The bastion accepted the connection because the public key was inserted into the AMI by Packer.

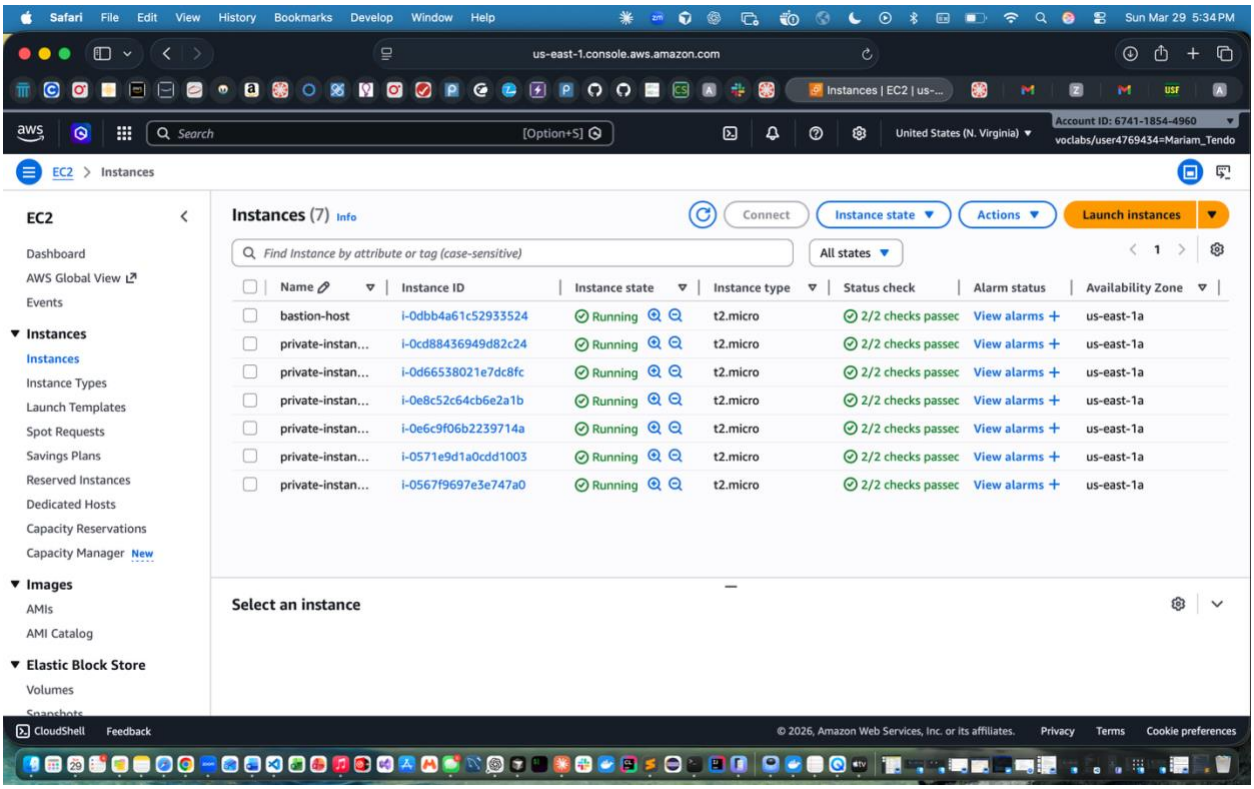
```
maryamtendo — ec2-user@ip-10-0-1-72:~ — ssh -i ~/.ssh/id_rsa ec2-us...

  "10.0.2.113",
  "10.0.2.131",
  "10.0.2.237",
  "10.0.2.213",
  "10.0.2.42",
]
vpc_id = "vpc-0b2affb9999a6b0c4"
bash-3.2$ ssh -i ~/.ssh/id_rsa ec2-user@3.91.215.61
The authenticity of host '3.91.215.61 (3.91.215.61)' can't be established.
ED25519 key fingerprint is SHA256:snrMew8uBVvtMf+J9ben8fIqhUlh/E7OSpcQghv2Ohk.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.91.215.61' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-1-72 ~]$ docker --version
```

- From the bastion host, I used SSH to connect into one of the private EC2 instances using its private IP address which successfully connected. Docker was also running on the private instance as well.

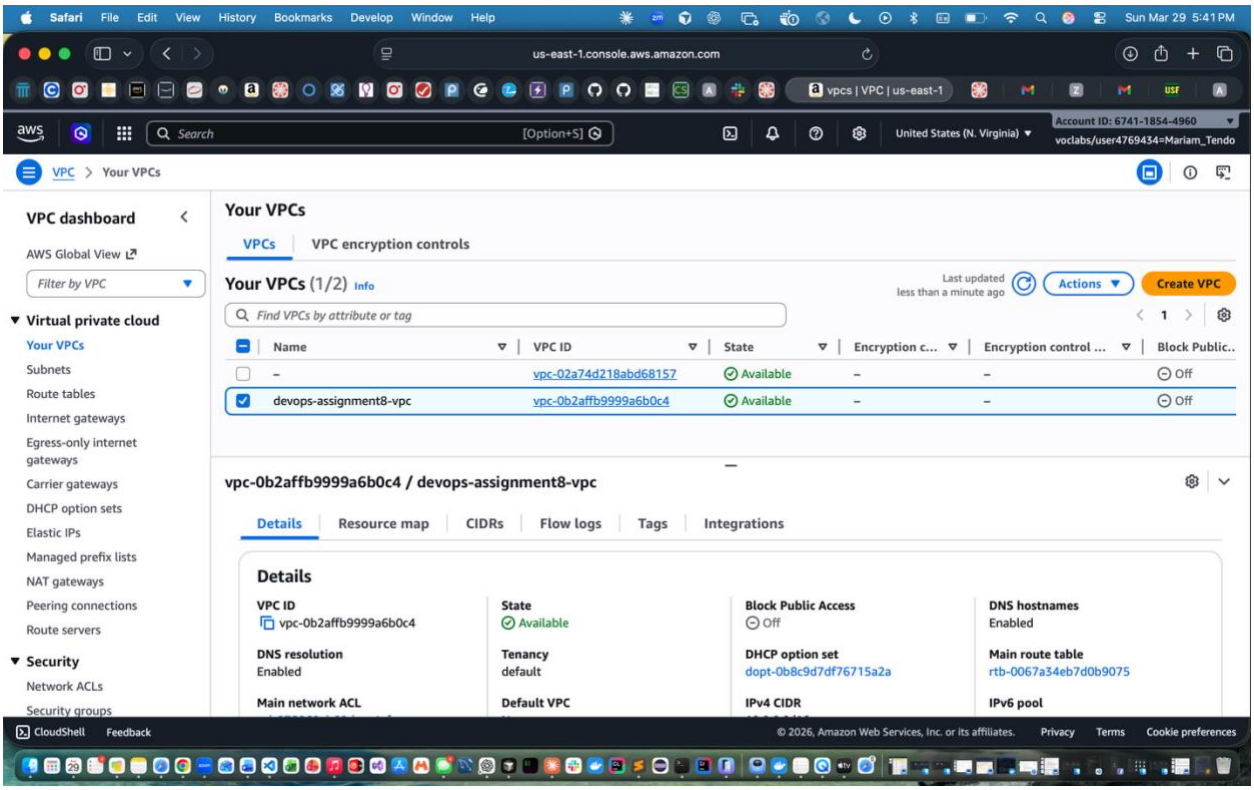
```
maryamtendo — ec2-user@ip-10-0-2-224:~ — ssh -i ~/.ssh/id_rsa ec2-...
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Mon Mar 30 00:22:25 2026 from 66.234.197.224
[[ec2-user@ip-10-0-1-72 ~]$ ssh -i ~/.ssh/id_rsa ec2-user@10.0.2.224            ]
The authenticity of host '10.0.2.224 (10.0.2.224)' can't be established.
ED25519 key fingerprint is SHA256:2BGNPkcbcOLLb5JNkg7iez1xhu26zDTTQilI3UnHQQ8.
This key is not known by any other names
[Are you sure you want to continue connecting (yes/no/[fingerprint])? yes       ]
Warning: Permanently added '10.0.2.224' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-2-224 ~]$
```

AWS Console Images to confirm

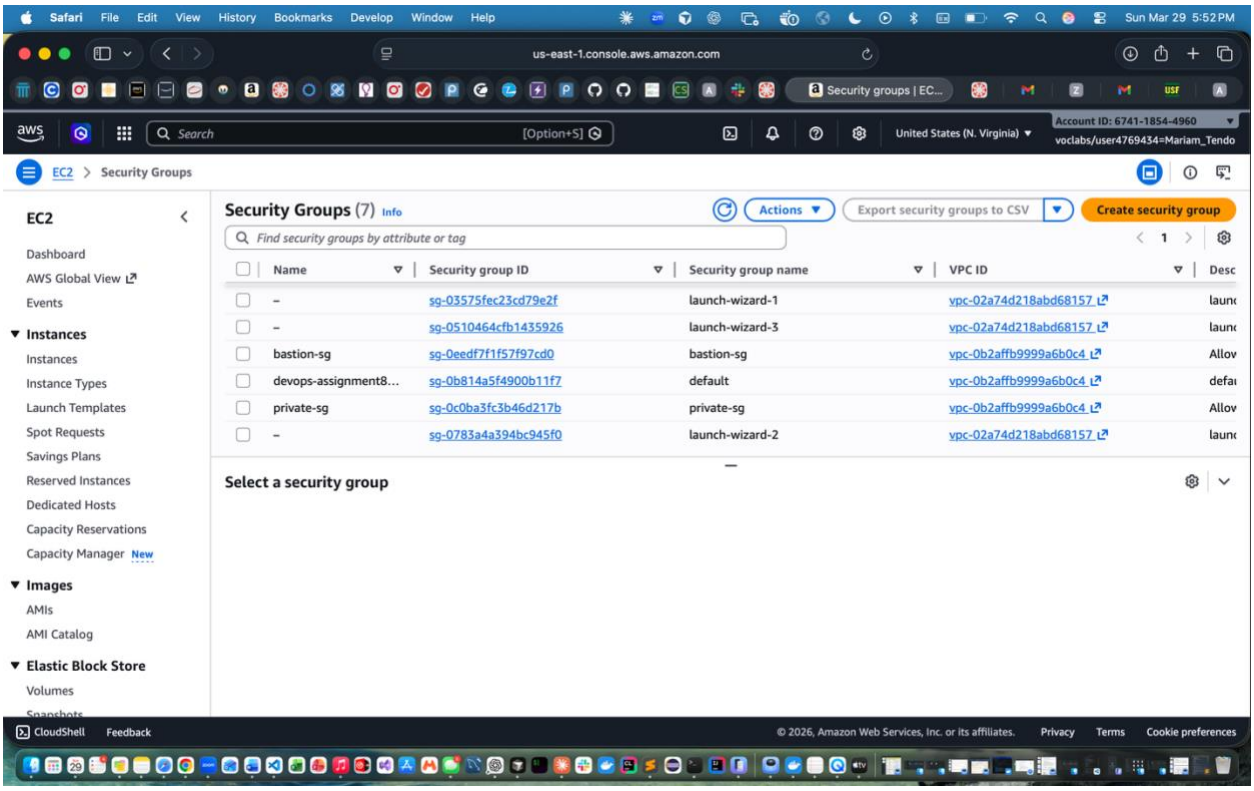
- **EC2 Instances (1 bastion + 6 private instances running)**



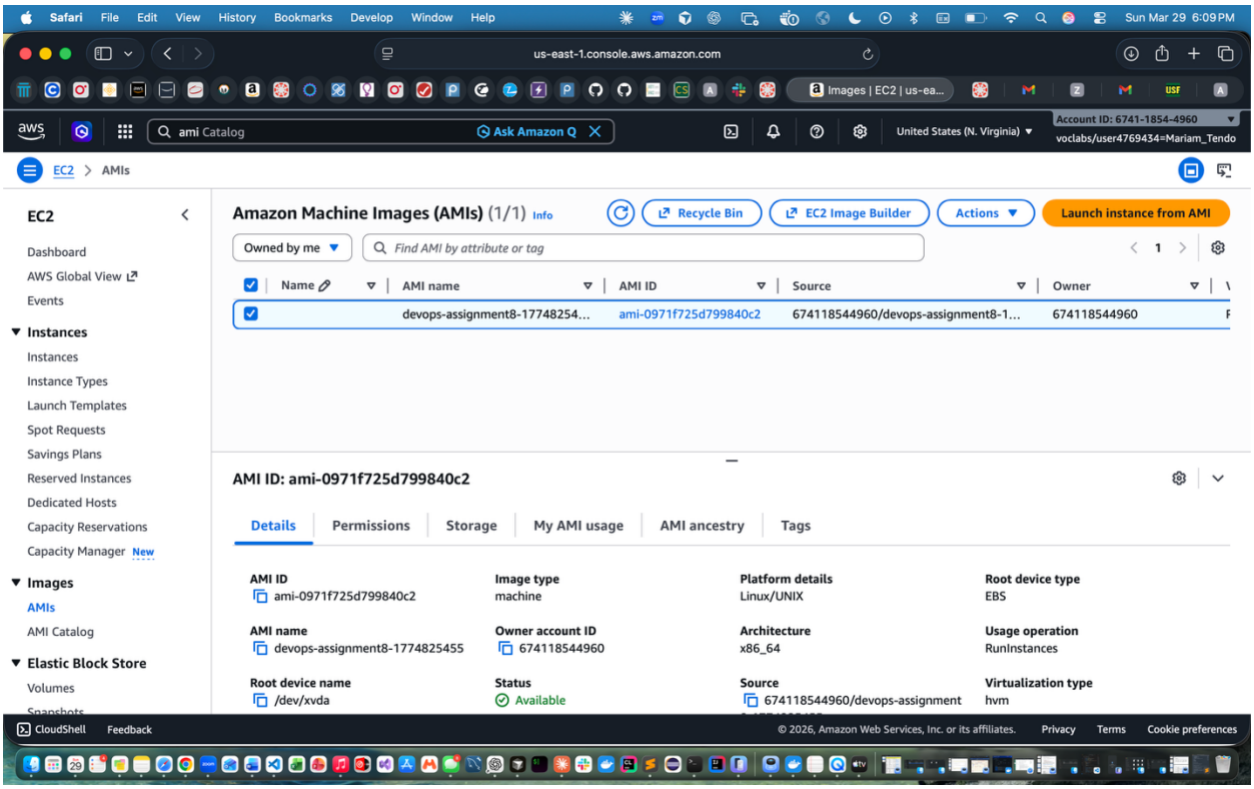
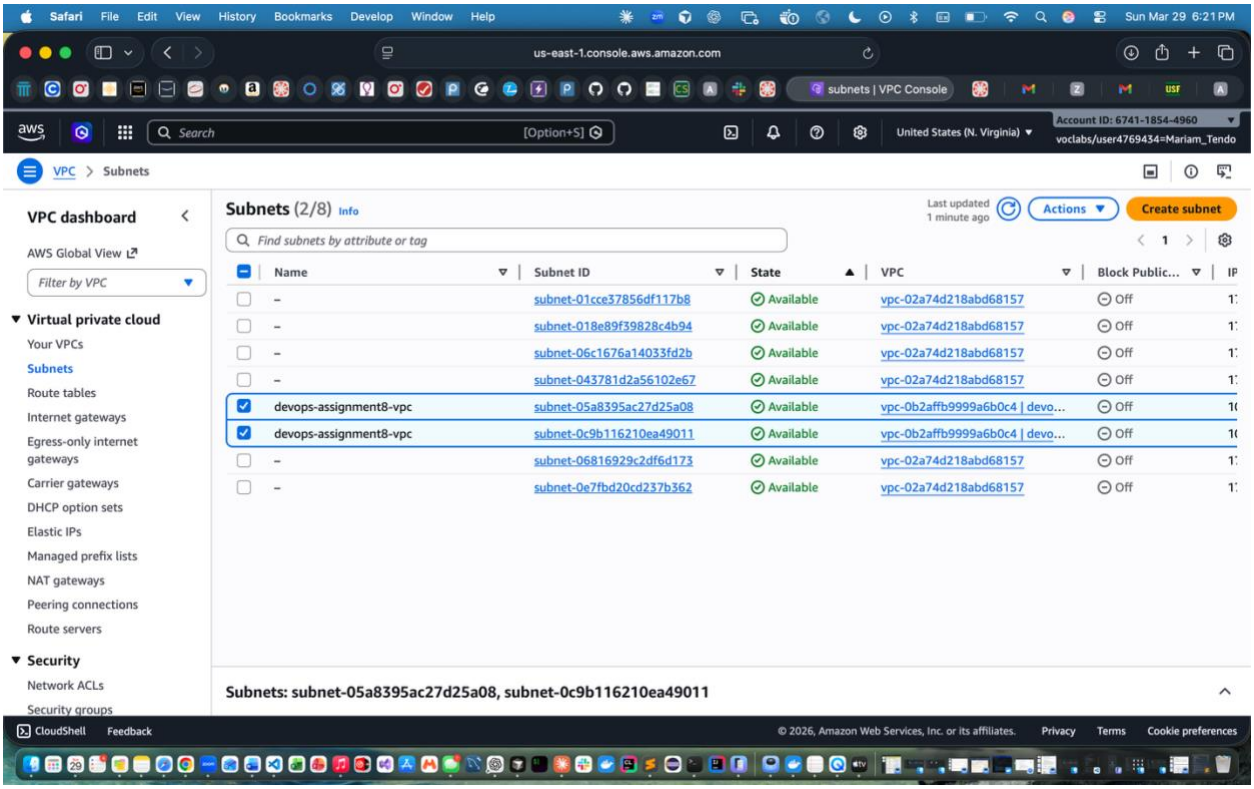
- **VPC (devops-assignment8-vpc)**



- **Security Groups (bastion-sg allows SSH from my IP only, private-sg allows SSH from bastion only)**



- **Custom AMI built by Packer (Amazon Linux , docker, SSH public key)**



- **Subnets (public subnet 10.0.1.0/24 and private subnet 10.0.2.0/24)**